

JobMatch AI

Product Requirements Document

Version 1.0 · February 2026

Status: Draft

1. Product Overview

JobMatch AI is a web application (with future mobile support) that helps job seekers find relevant opportunities by deeply understanding their skills, experience, and qualities — not just their job title. Unlike traditional job boards that rely on keyword matching, JobMatch AI uses AI-powered semantic analysis to surface roles a user is genuinely qualified for, including non-obvious matches they would never find on their own.

1.1 Problem Statement

Today's job search is broken in three key ways:

- Job boards match on titles and keywords, not transferable skills or actual accomplishments.
- Seekers must manually search multiple boards (LinkedIn, Indeed, Glassdoor, etc.), creating fragmented and tedious workflows.
- There is no system that explains why a job is or isn't a good match, leaving applicants guessing.

1.2 Vision

"Your resume should find jobs for you — not the other way around."

JobMatch AI acts as a personal recruiting assistant: it understands what you're actually good at, hunts across every major job board, and surfaces the best opportunities with clear, honest explanations of fit.

1.3 Target Users

- Primary: Active job seekers who apply to 10+ roles per month
 - Secondary: Passive job seekers exploring a career change or pivot
 - Future: Hiring teams and recruiters (out of scope for v1)
-

2. Goals & Success Metrics

2.1 Product Goals

- Reduce time-to-application by surfacing pre-matched, pre-explained job recommendations daily.
- Increase application quality by helping users understand and communicate their fit.
- Delight users with match explanations that feel genuinely insightful, not generic.

2.2 Success Metrics

- User applies to at least 1 recommended job within first session (activation)
 - $\geq 70\%$ of recommended jobs rated 3+ stars by user (match quality)
 - $\geq 40\%$ of users return within 7 days (retention)
 - Time from signup to first job application < 10 minutes
-

3. Feature Requirements

3.1 Resume Ingestion

Users must be able to provide their resume in two ways:

- **Upload a PDF or Word (.docx) file**
 - System extracts text, structure, and formatting using a parsing library (e.g., pdfjs, mammoth)
 - Supports multi-page resumes up to 10MB
- **Paste resume text directly**
 - Plain text input with a large text area
 - Basic markdown formatting preserved where possible

After ingestion, the AI parses the resume into a structured profile:

- Extracted skills (technical, soft, domain-specific)
- Job history with inferred impact and seniority
- Education and certifications
- Inferred strengths and career trajectory
- Preferred industries and role types (editable by user)

3.2 Job Search & Aggregation

Phase 1 — Web Scraping

The app will scrape job listings from public job boards. Target sources include:

- Indeed
- LinkedIn Jobs
- Glassdoor
- ZipRecruiter
- Wellfound (AngelList) — for startups
- Dice — for tech roles
- Google Jobs (via structured data)
- Company career pages (targeted scraping)

Scraping architecture considerations:

- Use rotating proxies and rate limiting to avoid blocks
- Jobs normalized into a standard schema: title, company, location, description, salary, posting date, source URL
- Deduplication across sources by hashing title + company + location
- Refresh cycle: new jobs every 6 hours per source

Phase 2 — API Integrations (Future)

Once the scraping layer is stable, the system will layer in official API access for reliability and richer data:

- JSearch API (aggregates Indeed, LinkedIn, Glassdoor, and others)
- Adzuna API
- Greenhouse and Lever APIs for direct company ATS listings

3.3 AI Matching Engine

This is the core differentiator of JobMatch AI. When a job is retrieved, the AI analyzes it against the user's profile using semantic understanding, not keyword overlap.

Match Score

- Each job receives a match score from 0–100
- Score is based on: skills alignment, experience level match, domain relevance, trajectory fit, and location/remote preference
- Scores are recalculated whenever the user updates their profile

Match Explanation

Every job surfaces a plain-language explanation covering:

- Why it's a strong match (specific skills, experience, or qualities that align)

- Potential gaps or concerns (honest assessment of missing qualifications)
- Transferable factors — qualities from different contexts that apply

Match Tiers

-  Strong Match (score 75–100) — High confidence, apply now
-  Good Match (score 50–74) — Solid fit with minor gaps
-  Reach (score 30–49) — Aspirational; worth applying with a strong cover letter
-  Hidden Gem — AI-flagged non-obvious matches where title differs but role aligns

3.4 Application Tracker (Kanban)

A built-in lightweight tracker so users never lose track of where they've applied:

- Columns: Saved → Applied → Phone Screen → Interview → Offer → Rejected
- One-click move a job from the results feed into the tracker
- Each card shows: company, role, match score, date applied, notes field
- Filter/sort by status, date, match score, or company
- Basic analytics: application funnel, response rate, avg time-to-response

3.5 Cover Letter Generator

For any job in their tracker or results, users can auto-generate a tailored cover letter:

- AI drafts the letter using the job description + user's resume profile
- Highlights the most relevant experiences for that specific role
- Editable in-app before copying or downloading as PDF/Word
- Tone options: formal, conversational, startup-casual
- User can save a preferred template/style that persists across generations

3.6 User Accounts & Authentication

- Sign up / log in via email + password, or OAuth (Google)
- All data (resume profile, tracker, preferences) stored in cloud and synced across devices
- Users can maintain multiple resume profiles (e.g., for different career tracks)
- Account deletion removes all data

3.7 Daily Digest (Future — v1.1)

- Daily or weekly email summary of new top matches
- User configures frequency and minimum match score threshold
- Push notifications for mobile (post-PWA upgrade)

4. Technical Architecture

4.1 Proposed Stack

- Frontend: React (web), designed PWA-ready for future mobile
- Backend: Node.js + Express or Python + FastAPI
- Database: PostgreSQL (user data, job cache) + Redis (job queue, caching)
- AI/LLM: Claude API (Anthropic) for resume parsing, job matching, and cover letter generation
- Scraping: Playwright or Puppeteer with rotating proxy support
- Auth: Supabase Auth or Auth0
- Storage: AWS S3 or Supabase Storage (uploaded resumes)
- Hosting: Vercel (frontend) + Railway or Render (backend)

4.2 Data Model (High Level)

- users — id, email, auth, created_at
- resume_profiles — id, user_id, raw_text, parsed_json, created_at
- jobs — id, title, company, location, description, source_url, source, salary_range, posted_at, scraped_at
- job_matches — id, user_id, job_id, score, explanation_json, tier, created_at
- applications — id, user_id, job_id, status, applied_at, notes, cover_letter

4.3 Scraping Architecture

- Scraper runs as a background job/worker, not in the web request cycle
- Job queue (Redis/BullMQ) manages scrape tasks per source
- Results normalized and deduplicated before insertion into jobs table
- Scraping is user-initiated on first login, then on a schedule
- Respect robots.txt; include fallback to API tier when scraping is blocked

5. Development Roadmap

Phase	Name	Deliverables	Timeline
Phase 0	Foundation	Project setup, auth, resume upload + parsing, basic profile UI	Weeks 1–2
Phase 1	Job Scraping	Scrapers for Indeed, LinkedIn, Glassdoor; job normalization; deduplication	Weeks 3–5
Phase 2	AI Matching	Match scoring, explanation generation, match tiers, hidden gem detection	Weeks 6–8
Phase 3	Tracker + Cover Letter	Kanban application tracker, cover letter generator, tone options	Weeks 9–11
Phase 4	Polish & Launch	UI polish, performance, onboarding flow, analytics, beta launch	Weeks 12–14
Phase 5	API Tier	Layer in JSearch / Adzuna APIs, more job sources, improved freshness	Post-launch
Phase 6	Mobile	PWA upgrade or React Native app, push notifications, mobile-optimized UX	Post-launch

6. Non-Functional Requirements

- Performance: Job results page loads in < 2 seconds; match scores computed within 5 seconds of profile update
- Scalability: Architecture supports 10,000 concurrent users without redesign
- Privacy: Resumes stored encrypted at rest; users can delete all data at any time
- Reliability: 99.5% uptime SLA for core matching and tracker features
- Accessibility: WCAG 2.1 AA compliance for all core flows
- Mobile-ready: Responsive design from day one; PWA installable in Phase 4

7. Out of Scope (v1)

- Recruiter-facing tools or candidate sourcing
- Interview prep or coaching features
- Salary negotiation assistant
- Direct ATS integration (auto-submitting applications)
- Resume editing or rewriting tools

- Social/referral network features
-

8. Open Questions & Risks

- Scraping fragility: Job boards actively block scrapers. Need a plan for when scraping breaks (API fallback, user notification).
 - LLM cost at scale: Claude API calls per match could become expensive. Need caching strategy — match explanations should be generated once and stored, not regenerated per request.
 - Resume parsing accuracy: Edge cases in PDF parsing (images, tables, two-column layouts) may produce poor extractions. May need a manual edit step post-parse.
 - LinkedIn ToS: LinkedIn explicitly prohibits scraping. We may need to either rely on third-party LinkedIn data aggregators or prompt users to paste job descriptions manually for LinkedIn roles.
 - Job freshness: Scraped jobs may be stale or filled. Consider surfacing posting date prominently and auto-archiving jobs older than 30 days.
-